

Assignment-1

Name: Rikhil Nellimarla

Registration Number: 23BEC7030

1. Kth Smallest element

<https://www.hackerrank.com/contests/algorithms-2-1/challenges/iiitsalgoq1>

Only available in C/C++

2. Find Equilibrium Point

<https://www.codechef.com/practice/course/tcs-nqt-questions/TCSNQT/problems/TCSNQTCP09>

```
import java.io.*;
import java.util.*;

class Main {
    public static void main(String[] args) throws Exception {
        FastScanner fs = new FastScanner(System.in);
        StringBuilder out = new StringBuilder();

        int T = fs.nextInt();
        while (T-- > 0) {
            int N = fs.nextInt();
            int[] nums = new int[N];

            long totalSum = 0;
            for (int i = 0; i < N; i++) {
                nums[i] = fs.nextInt();
                totalSum += nums[i];
            }

            long leftSum = 0;
            int answer = -1;

            for (int i = 0; i < N; i++) {
                long rightSum = totalSum - leftSum - nums[i];
                if (leftSum == rightSum) {
                    answer = i;
                    break; // leftmost equilibrium index
                }
                leftSum += nums[i];
            }
        }
    }
}
```

```

        out.append(answer).append('\n');
    }

    System.out.print(out.toString());
}

// Fast input reader (essential for CodeChef)
static class FastScanner {
    private final byte[] buffer = new byte[1 << 16];
    private int ptr = 0, len = 0;
    private final InputStream in;

    FastScanner(InputStream in) {
        this.in = in;
    }

    private int readByte() throws IOException {
        if (ptr >= len) {
            len = in.read(buffer);
            ptr = 0;
            if (len <= 0) return -1;
        }
        return buffer[ptr++];
    }

    int nextInt() throws IOException {
        int c, sign = 1, val = 0;
        do {
            c = readByte();
        } while (c <= ' ');

        if (c == '-') {
            sign = -1;
            c = readByte();
        }

        while (c > ' ') {
            val = val * 10 + (c - '0');
            c = readByte();
        }
        return val * sign;
    }
}

```

Output:

✓ You got it right!

AI Review?

Sub-Task	Task #	Result (time)
1	0	Correct (0.04)
Subtask Score: 25%		Result - Correct
2	1	Correct (0.03)
Subtask Score: 25%		Result - Correct
3	2	Correct (0.03)
Subtask Score: 25%		Result - Correct
4	3	Correct (0.02)
Subtask Score: 25%		Result - Correct
Total Score = 100%		

3. Backtracking - Find Unique Permutations

<https://www.codechef.com/practice/course/flipkart-interview-questions/FLIPKARTPREP/problems/PREP24>

```
import java.io.*;
import java.util.*;

class Main {

    static List<int[]> result;
    static int[] arr;
    static boolean[] used;
    static int n;

    public static void main(String[] args) throws Exception {
        FastScanner fs = new FastScanner(System.in);
        StringBuilder out = new StringBuilder();
```

```

int T = fs.nextInt();
while (T-- > 0) {
    n = fs.nextInt();
    arr = new int[n];
    for (int i = 0; i < n; i++) {
        arr[i] = fs.nextInt();
    }

    Arrays.sort(arr); // crucial for lexicographic order
    used = new boolean[n];
    result = new ArrayList<>();

    backtrack(new int[n], 0);

    out.append(result.size()).append('\n');
    for (int[] perm : result) {
        for (int x : perm) {
            out.append(x).append(' ');
        }
        out.append('\n');
    }
}

System.out.print(out.toString());
}

static void backtrack(int[] curr, int idx) {
    if (idx == n) {
        result.add(curr.clone());
        return;
    }

    for (int i = 0; i < n; i++) {
        if (used[i]) continue;

        // skip duplicates
        if (i > 0 && arr[i] == arr[i - 1] && !used[i - 1]) continue;

        used[i] = true;
        curr[idx] = arr[i];
        backtrack(curr, idx + 1);
        used[i] = false;
    }
}

// Fast input (mandatory for CodeChef)
static class FastScanner {
    private final byte[] buffer = new byte[1 << 16];
    private int ptr = 0, len = 0;
    private final InputStream in;

```

```

FastScanner(InputStream in) {
    this.in = in;
}

private int readByte() throws IOException {
    if (ptr >= len) {
        len = in.read(buffer);
        ptr = 0;
        if (len <= 0) return -1;
    }
    return buffer[ptr++];
}

int nextInt() throws IOException {
    int c, sign = 1, val = 0;
    do {
        c = readByte();
    } while (c <= ' ');

    if (c == '-') {
        sign = -1;
        c = readByte();
    }

    while (c > ' ') {
        val = val * 10 + (c - '0');
        c = readByte();
    }
    return val * sign;
}
}
}

```

Output:

✓ Excellent work!

AI Review?

Sub-Task	Task #	Result (time)
1	0	Correct (0.26)
Subtask Score: 25%		Result - Correct
2	1	Correct (0.10)
Subtask Score: 25%		Result - Correct
3	2	Correct (0.03)
Subtask Score: 25%		Result - Correct
4	3	Correct (0.06)
Subtask Score: 25%		Result - Correct
Total Score = 100%		

4. Mutated Minions

<https://www.codechef.com/problems/CHN15A?tab=statement>

```
import java.io.*;
import java.util.*;

class Main {
    public static void main(String[] args) throws Exception {
        FastScanner fs = new FastScanner(System.in);
        StringBuilder sb = new StringBuilder();

        int T = fs.nextInt();
        while (T-- > 0) {
            int N = fs.nextInt();
            int K = fs.nextInt();
            int count = 0;

            for (int i = 0; i < N; i++) {
```

```

        int val = fs.nextInt();
        if ((val + K) % 7 == 0) {
            count++;
        }
    }
    sb.append(count).append('\n');
}
System.out.print(sb.toString());
}

static class FastScanner {
    private final byte[] buffer = new byte[1 << 16];
    private int ptr = 0, len = 0;
    private final InputStream in;

    FastScanner(InputStream in) {
        this.in = in;
    }

    private int readByte() throws IOException {
        if (ptr >= len) {
            len = in.read(buffer);
            ptr = 0;
            if (len <= 0) return -1;
        }
        return buffer[ptr++];
    }

    int nextInt() throws IOException {
        int c, sign = 1, val = 0;
        do {
            c = readByte();
        } while (c <= ' ');

        if (c == '-') {
            sign = -1;
            c = readByte();
        }

        while (c > ' ') {
            val = val * 10 + (c - '0');
            c = readByte();
        }
        return val * sign;
    }
}

```

Output:

✓ Correct Answer

Submission ID: [1224404712](#)

Sub-Task	Task #	Result (time)
1	0	Correct (0.02)
Subtask Score: 100%		Result - Correct
Total Score = 100%		

5. Sums in a Triangle

<https://www.codechef.com/problems/SUMTRIAN>

```
import java.io.*;
import java.util.*;

class Main {
    public static void main(String[] args) throws Exception {
        FastScanner fs = new FastScanner(System.in);
        StringBuilder sb = new StringBuilder();

        int T = fs.nextInt();
        while (T-- > 0) {
            int N = fs.nextInt();
            int[][] dp = new int[N][N];

            for (int i = 0; i < N; i++) {
                for (int j = 0; j <= i; j++) {
                    dp[i][j] = fs.nextInt();
                }
            }

            for (int i = N - 2; i >= 0; i--) {
                for (int j = 0; j <= i; j++) {
                    dp[i][j] += Math.max(dp[i + 1][j], dp[i + 1][j + 1]);
                }
            }

            sb.append(dp[0][0]).append('\n');
        }
    }
}
```

```

        System.out.print(sb.toString());
    }

    static class FastScanner {
        private final byte[] buffer = new byte[1 << 16];
        private int ptr = 0, len = 0;
        private final InputStream in;

        FastScanner(InputStream in) {
            this.in = in;
        }

        private int readByte() throws IOException {
            if (ptr >= len) {
                len = in.read(buffer);
                ptr = 0;
                if (len <= 0) return -1;
            }
            return buffer[ptr++];
        }

        int nextInt() throws IOException {
            int c, sign = 1, val = 0;
            do {
                c = readByte();
            } while (c <= ' ');

            if (c == '-') {
                sign = -1;
                c = readByte();
            }

            while (c > ' ') {
                val = val * 10 + (c - '0');
                c = readByte();
            }
            return val * sign;
        }
    }
}

```

Output:

✓ Correct Answer

Submission ID: [1224404819](#)

Sub-Task	Task #	Result (time)
1	0	Correct (0.10)
Subtask Score: 100%		Result - Correct
Total Score = 100%		

6. Dominant Army

<https://www.codechef.com/problems/DOMINANT>

```
import java.io.*;
import java.util.*;

class Main {
    public static void main(String[] args) throws Exception {
        FastScanner fs = new FastScanner(System.in);
        StringBuilder sb = new StringBuilder();

        int T = fs.nextInt();
        while (T-- > 0) {
            int NA = fs.nextInt();
            int NB = fs.nextInt();
            int NC = fs.nextInt();

            if (NA > NB + NC || NB > NA + NC || NC > NA + NB) {
                sb.append("YES\n");
            } else {
                sb.append("NO\n");
            }
        }

        System.out.print(sb.toString());
    }

    static class FastScanner {
        private final byte[] buffer = new byte[1 << 16];
        private int ptr = 0, len = 0;
        private final InputStream in;
```

```

FastScanner(InputStream in) {
    this.in = in;
}

private int readByte() throws IOException {
    if (ptr >= len) {
        len = in.read(buffer);
        ptr = 0;
        if (len <= 0) return -1;
    }
    return buffer[ptr++];
}

int nextInt() throws IOException {
    int c, sign = 1, val = 0;
    do {
        c = readByte();
    } while (c <= ' ');

    if (c == '-') {
        sign = -1;
        c = readByte();
    }

    while (c > ' ') {
        val = val * 10 + (c - '0');
        c = readByte();
    }
    return val * sign;
}
}
}

```

Output:

Sub-Task	Task #	Result (time)
1	0	Correct (0.03)
Subtask Score: 20%		Result - Correct
2	1	Correct (0.02)
Subtask Score: 20%		Result - Correct
3	2	Correct (0.03)
Subtask Score: 20%		Result - Correct
4	3	Correct (0.02)
Subtask Score: 20%		Result - Correct
5	4	Correct (0.02)
Subtask Score: 20%		Result - Correct
Total Score = 100%		

7. Most Frequent Element

<https://www.hackerrank.com/contests/bits-hyderabad-practice-test-1/challenges/most-frequent-element>

```
import java.io.*;
import java.util.*;

public class Solution {
    public static void main(String[] args) throws Exception {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        int N = Integer.parseInt(br.readLine().trim());

        HashMap<Long, Integer> freq = new HashMap<>();
        for (int i = 0; i < N; i++) {
            long x = Long.parseLong(br.readLine().trim());
            freq.put(x, freq.getDefault(x, 0) + 1);
        }

        long answer = Long.MAX_VALUE;
```

```

int maxFreq = 0;

for (Map.Entry<Long, Integer> entry : freq.entrySet()) {
    long key = entry.getKey();
    int value = entry.getValue();

    if (value > maxFreq || (value == maxFreq && key < answer)) {
        maxFreq = value;
        answer = key;
    }
}

System.out.println(answer);
}
}

```

Output:

Most Frequent Element!

locked

Problem

Submissions

Leaderboard

Discussions

Submitted a few seconds ago • Score: 100.00

Status: **Accepted**



Test Case #0



Test Case #1



Test Case #2



Test Case #3



Test Case #4

8. Quicksort In-Place

<https://www.hackerrank.com/challenges/quicksort3/problem>

```

import java.io.*;
import java.util.*;

public class Solution {

    static int[] arr;

    public static void main(String[] args) throws Exception {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));

        int n = Integer.parseInt(br.readLine().trim());
        arr = new int[n];
    }
}

```

```

StringTokenizer st = new StringTokenizer(br.readLine());
for (int i = 0; i < n; i++) {
    arr[i] = Integer.parseInt(st.nextToken());
}

quickSort(0, n - 1);
}

static void quickSort(int low, int high) {
    if (low >= high) return;

    int p = partition(low, high);
    printArray();

    quickSort(low, p - 1);
    quickSort(p + 1, high);
}

static int partition(int low, int high) {
    int pivot = arr[high];
    int i = low;

    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            swap(i, j);
            i++;
        }
    }
    swap(i, high);
    return i;
}

static void swap(int i, int j) {
    int temp = arr[i];
    arr[i] = arr[j];
    arr[j] = temp;
}

static void printArray() {
    StringBuilder sb = new StringBuilder();
    for (int x : arr) {
        sb.append(x).append(" ");
    }
    System.out.println(sb.toString().trim());
}
}

```

Output:

✓ **Test case 0**

✓ Test case 1

✓ Test case 2

✓ Test case 3

Compiler Message

Success

Input (stdin)

Download

1	7
2	1 3 9 8 2 7 5

Expected Output

Download

1	1 3 2 5 9 7 8
2	1 2 3 5 9 7 8
3	1 2 3 5 7 8 9

9. Maneuvering a Cave

<https://www.codechef.com/practice/course/tcs-interview-questions/CODEVITA2020/problems/CAVEPATH>

```
import java.io.*;
import java.util.*;

class Main {
    static final long MOD = 1000000007L;

    public static void main(String[] args) throws Exception {
        FastScanner fs = new FastScanner(System.in);
        StringBuilder sb = new StringBuilder();

        int T = fs.nextInt();
        while (T-- > 0) {
            int N = fs.nextInt();
            int M = fs.nextInt();

            long[][] dp = new long[N][M];

            for (int i = 0; i < N; i++) {
                dp[i][0] = 1;
            }
            for (int j = 0; j < M; j++) {
                dp[0][j] = 1;
            }
        }
    }
}
```

```

        for (int i = 1; i < N; i++) {
            for (int j = 1; j < M; j++) {
                dp[i][j] = (dp[i - 1][j] + dp[i][j - 1]) % MOD;
            }
        }

        sb.append(dp[N - 1][M - 1]).append('\n');
    }

    System.out.print(sb.toString());
}

static class FastScanner {
    private final byte[] buffer = new byte[1 << 16];
    private int ptr = 0, len = 0;
    private final InputStream in;

    FastScanner(InputStream in) {
        this.in = in;
    }

    private int readByte() throws IOException {
        if (ptr >= len) {
            len = in.read(buffer);
            ptr = 0;
            if (len <= 0) return -1;
        }
        return buffer[ptr++];
    }

    int nextInt() throws IOException {
        int c, sign = 1, val = 0;
        do {
            c = readByte();
        } while (c <= ' ');

        if (c == '-') {
            sign = -1;
            c = readByte();
        }

        while (c > ' ') {
            val = val * 10 + (c - '0');
            c = readByte();
        }
        return val * sign;
    }
}

```

Output:

✓ Awesome, you nailed it!		AI Review?
Sub-Task	Task #	Result (time)
1	0	Correct (0.02)
Subtask Score: 50%		Result - Correct
2	1	Correct (0.04)
Subtask Score: 50%		Result - Correct
Total Score = 100%		

10. Tram ride

<https://www.hackerearth.com/practice/data-structures/arrays/1-d/practice-problems/algorithm/tram-ride-d7ff3a92/?purpose=login&source=problem-page&update=google>

```
static long solve(int N, int start, int finish, int[] Ticket_cost){  
  
    if (start == finish) return 0;  
  
    start--; // convert to 0-based  
    finish--;  
  
    long clockwise = 0;  
  
    int i = start;  
  
    while (i != finish) {
```

```
        clockwise += Ticket_cost[i];

        i = (i + 1) % N;
    }

    long anticlockwise = 0;

    i = start;

    while (i != finish) {

        i = (i - 1 + N) % N;

        anticlockwise += Ticket_cost[i];

    }

    return Math.min(clockwise, anticlockwise);

}

}
```

Output:

RESULT:  Accepted

 [Refer judge environment](#)

Score	Time (sec)	Memory (KiB)	Language
0	0.61503	93572	Java 17

Input	Result	Time (sec)	Memory (KiB)	Score	Your output	Correct output	Diff
Input #1	 Accepted	0.050331	2	1			
Input #2	 Accepted	0.050444	2	1			
Input #3	 Accepted	0.050459	92028	1			
Input #4	 Accepted	0.050319	2	1			
Input #5	 Accepted	0.050353	92060	1			
Input #6	 Accepted	0.050382	92032	3			
Input #7	 Accepted	0.050455	91076	3			
Input #8	 Accepted	0.058602	92032	3			
Input #9	 Accepted	0.082835	2	3			
Input #10	 Accepted	0.120847	93572	3			